Solution: Longest Palindrome(easy)

Problem Statement:

Given a string, determine the length of the longest palindrome that can be constructed using the characters from the string. Return the maximum possible length of the palindromic string.

Examples:

1. - **Input:** "applepie"
   - **Expected Output:** 5
   - **Justification:** The longest palindrome that can be constructed from the string is "pepep", which has a length of 5. There are are other palindromes too but they all will be of length 5.
2. - **Input:** "aabbcc"
   - **Expected Output:** 6
   - **Justification:** We can form the palindrome "abccba" using the characters from the string, which has a length of 6.
3. - **Input:** "bananas"
   - **Expected Output:** 5
   - **Justification:** The longest palindrome that can be constructed from the string is "anana", which has a length of 5.

Solution

To solve this problem, we can use a hashmap to keep track of the frequency of each character in the string. The idea is to use pairs of characters to form the palindrome. For example, if a character appears an even number of times, we can use all of them in the palindrome. If a character appears an odd number of times, we can use all except one of them in the palindrome. Additionally, if there's any character that appears an odd number of times, we can use one of them as the center of the palindrome.

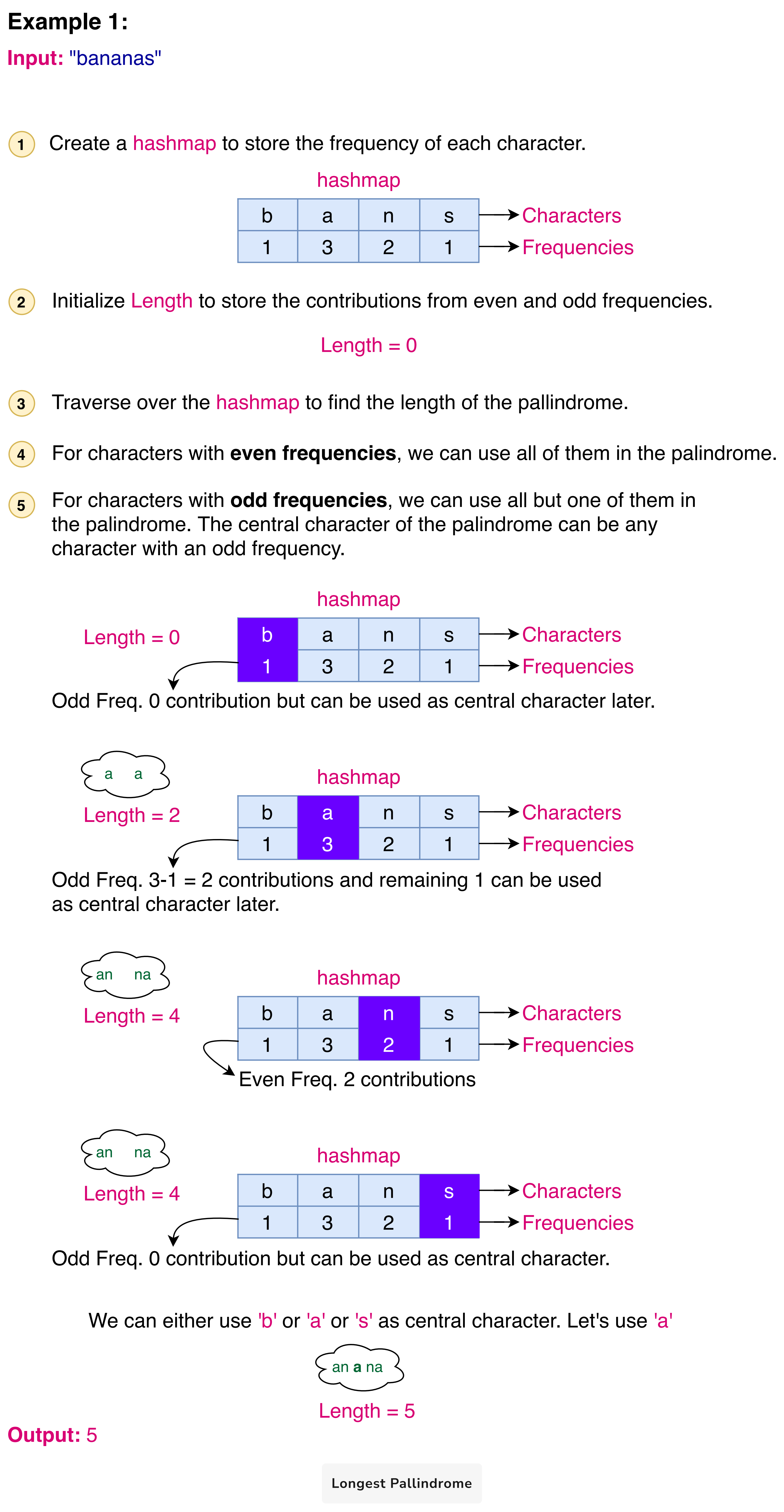
1. **Initialization:** Start by initializing a hashmap to keep track of the characters and their frequencies.
2. **Character Counting:** Iterate through the string and populate the hashmap with the frequency of each character.
3. **Palindrome Length Calculation:** For each character in the hashmap, if it appears an even number of times, add its count to the palindrome length. If it appears an odd number of times, add its count minus one to the palindrome length. Also, set a flag indicating that there's a character available for the center of the palindrome.
4. **Final Adjustment:** If the center flag is set, add one to the palindrome length.

Algorithm Walkthrough:

1. **Initialize a HashMap:**
   - We'll use a hashmap to store the frequency of each character in the string.
2. **Populate the HashMap:**
   - For the string "bananas", our hashmap will look like this:
     - b: 1
     - a: 3
     - n: 2
     - s: 1
3. **Determine Palindrome Length:**
   - We'll iterate through the hashmap to determine the length of the palindrome we can form.
4. **Even Frequencies:**
   - For characters with even frequencies, we can use all of them in the palindrome. For our string, the character 'n' has an even frequency.
     - 'n' can contribute 2 characters.
   - So far, we have a contribution of 2 characters to the palindrome.
5. **Odd Frequencies:**
   - For characters with odd frequencies, we can use all but one of them in the palindrome. The central character of the palindrome can be any character with an odd frequency.
   - For our string, characters 'b', 'a', and 's' have odd frequencies.
     - 'b' can contribute 0 characters (leaving out 1).
     - 'a' can contribute 2 characters (leaving out 1).
     - 's' can contribute 0 characters (leaving out 1).
   - Additionally, one of the characters left out from the odd frequencies can be used as the central character of the palindrome. Let's use 'a' for this purpose.
   - So, from the odd frequencies, we have a contribution of 2 characters to the palindrome, plus 1 for the central character.
6. **Total Length:**
   - Combining the contributions from even and odd frequencies, we get a total palindrome length of 2 (from even frequencies) + 3 (from odd frequencies) = 5.

The longest palindrome that can be constructed from "bananas" is of length 5.

Here is the visual representation of the algorithm:



Example 1:

Input: "bananas"

① Create a hashmap to store the frequency of each character.

② Initialize Length to store the contributions from even and odd frequencies.

③ Traverse over the hashmap to find the length of the pallindrome.

④ For characters with **even frequencies**, we can use all of them in the palindrome.

⑤ For characters with **odd frequencies**, we can use all but one of them in the palindrome. The central character of the palindrome can be any character with an odd frequency.

Odd Freq. 0 contribution but can be used as central character later.

Odd Freq. 3-1 = 2 contributions and remaining 1 can be used as central character later.

Even Freq. 2 contributions

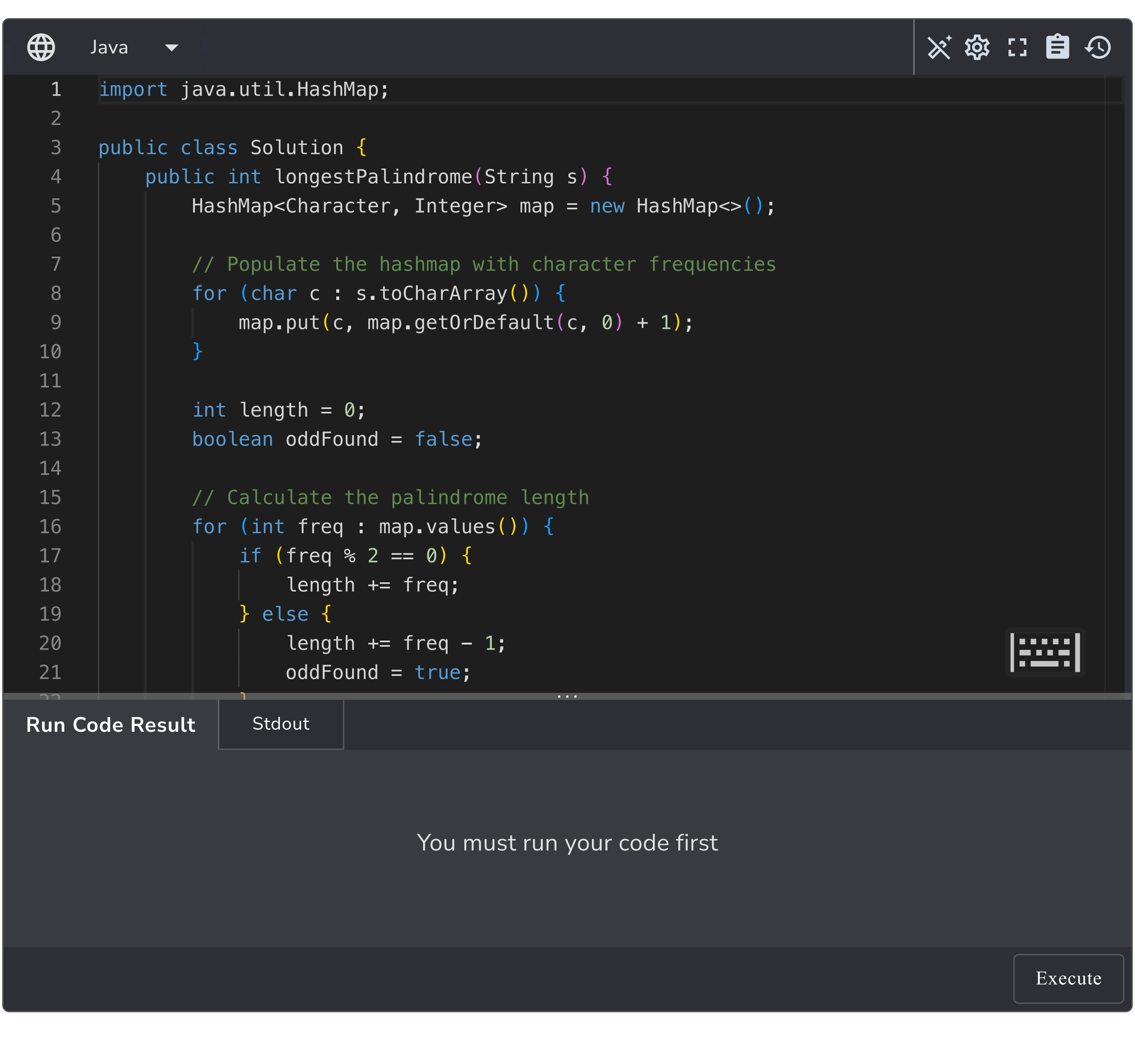
Odd Freq. 0 contribution but can be used as central character.

We can either use 'b' or 'a' or 's' as central character. Let's use 'a'

Output: 5

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;

public class Solution {
    public int longestPalindrome(String s) {
        HashMap<Character, Integer> map = new HashMap<>();

        // Populate the hashmap with character frequencies
        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        int length = 0;
        boolean oddFound = false;

        // Calculate the palindrome length
        for (int freq : map.values()) {
            if (freq % 2 == 0) {
                length += freq;
            } else {
                length += freq - 1;
                oddFound = true;
```

You must run your code first

Time Complexity:

1. **Iterating through the string**: We iterate through the entire string once to count the frequency of each character. This operation takes (O(n)) time, where (n) is the length of the string.
2. **Iterating through the hashmap**: After counting the frequencies, we iterate through the hashmap to determine how many characters can be used to form the palindrome. In the worst case, this would be (O(26)) for the English alphabet, which is a constant time. However, in general terms, if we consider any possible character (not just English alphabet), this would be (O(k)), where (k) is the number of unique characters in the string. But since $k <= n$, this can also be considered (O(n)) in the worst case.

Combining the two steps, the overall time complexity is (O(n) + O(n) = O(n)).

Space Complexity:

Hashmap for character frequencies: The space taken by the hashmap is proportional to the number of unique characters in the string. In the worst case, this would be (O(26)) for the English alphabet, which is a constant space. However, in general terms, if we consider any possible character (not just English alphabet), this would be (O(k)), where (k) is the number of unique characters in the string. But since $k <= n$, this can also be considered (O(n)) in the worst case.

Thus, the space complexity of the algorithm is (O(n)).

← Previous
Problem 4: Longest Palindrome(Easy)

Next →
Problem 5: Ransom Note (Easy)

☐ Mark as Completed